

Documentación de Diseño - Trabajo Práctico Integrador UNQ-Shop

Materia: Programación Orientada a Objetos - UNQ (2026)

Integrantes del Grupo

Nombre y Apellido	Correo Electrónico
Francisco Demartino	frandema19@gmail.com

1. Patrones de Diseño Utilizados y Roles (Gamma et. al.)

El sistema fue diseñado priorizando la alta cohesión y el bajo acoplamiento mediante la aplicación de 6 patrones de diseño del catálogo GoF:

Módulo 2.1 - Catálogo de Productos (Patrón COMPOSITE)

- **Propósito:** Tratar productos individuales y paquetes de manera uniforme para gestionar el catálogo y el cálculo de precios recursivo.
- **Roles GoF:**
 - *Componente:* Item (clase abstracta).
 - *hoja:* Producto (clase concreta).
 - *nodo:* Paquete (clase concreta que agrupa una lista de Item).

Módulo 2.2 - Ciclo de Vida del Pedido (Patrón STATE)

- **Propósito:** Permitir que un pedido modifique su comportamiento interno cuando su estado cambia, eliminando sentencias condicionales extensas durante el procesamiento.
- **Roles GoF:**

- *Contexto*: Pedido.
- *Estado*: Estado (clase abstracta).
- *estados concretos*: Borrador, Confirmado, En_Preparacion, Enviado, Entregado, Cancelado.

Módulo 2.3 - Métodos de Envío (Patrón STRATEGY)

- **Propósito**: Definir una familia de algoritmos de cálculo de envío, encapsulándolos y haciéndolos intercambiables dentro del pedido.
- **Roles GoF**:
 - *Contexto*: Pedido.
 - *estrategia*: EstrategiaDeEnvio (interfaz).
 - *estrategias concretas*: Estándar, Express, Sucursal.

Módulo 2.4 - Métodos de Pago (Patrón TEMPLATE METHOD)

- **Propósito**: Definir el esqueleto del algoritmo de pago (validar, reservar, transferir, notificar), permitiendo que las subclasses redefinan ciertos pasos sin cambiar la estructura del proceso.
- **Roles GoF**:
 - *clase abstracta*: MedioDePago (define el método público procesarPago()).
 - *Clases concretas*: Tarjeta, Transferencia, BilleteraVirtual (implementan las operaciones que MedioDePago les dicta).

Módulo 2.5 - Notificaciones (Patrón OBSERVER)

- **Propósito**: Definir una dependencia de uno-a-muchos para que cuando el Pedido cambie de estado, todos los subsistemas interesados sean notificados automáticamente.
- **Roles GoF**:
 - *Subject*: Observable (superclase de la que hereda Pedido).
 - *Observador*: Observador (interfaz).
 - *observadores concretos*: NotificadorEmail, Fidelizacion, GeneradorFactura.

Módulo 2.6 - Búsqueda en el Catálogo (Patrones STRATEGY + COMPOSITE)

- **Propósito:** Permitir filtrar el catálogo aplicando criterios individuales o combinados mediante operadores lógicos en tiempo de ejecución.
- **Roles GoF:**
 - *estrategia* / *Componente*: Criterio (interfaz base).
 - *estrategias concretas* / *hojas*: PorNombre, PorPrecioMaximo, PorCategoria, PorDisponibilidad.
 - *nodos* : And, Or (agrupan listas de criterios), Not (envuelve un solo criterio).

Módulo 2.8 - Reportes (Patrón VISITOR con Double Dispatch)

- **Propósito:** Representar una operación que se realiza sobre los elementos del catálogo, permitiendo agregar nuevos tipos de reportes sin modificar las clases de los productos o paquetes.
- **Roles GoF:**
 - *Visitor*: ReporteVisitor (interfaz).
 - *visitor concreto*: ReporteDeProductosMasVendidos.
 - *Element*: VisitablePorReporte (interfaz base para Item).
 - *elementos concretos*: Producto, Paquete.
 - *Object Structure*: Catalogo (actúa como orquestador del recorrido).

2. Decisiones de Diseño y Detalles de Implementación Relevantes

Durante el desarrollo de la capa de dominio, se tomaron decisiones arquitectónicas fundamentadas en principios SOLID y *Clean Code*:

1. **Gestión de Stock en Depósito:** En la clase Depósito, se implementó una fase de validación pidiendo primero los ítems requeridos mediante el agrupamiento de ítems repetidos en la lista de ítems para agruparlos en un Map<Item,cantidad> . Esto evita inconsistencias en el inventario, asegurando que el pedido solo avance a Confirmado si la totalidad del carrito está garantizada.

2. **Transmisión de Contexto en el Observer:** Al invocar `notificarObservadores`, el Pedido se envía a sí mismo junto con el Estado viejo y el Estado nuevo. Esta optimización permite que los observadores decidan dinámicamente si reaccionar al evento o no por ellos mismos.
3. **Rol del Catálogo en el Visitor:** La clase `Catalogo` actúa como *Object Structure*. Su responsabilidad es actuar como director de orquesta, iterando sobre la colección e invocando recursivamente el `accept` sobre cada ítem, sin implementar ella misma la interfaz `VisitablePorReporte`.
4. **Principio DRY(dont repeat yourself) en Reportes:** Para el `ReporteDeProductosMasVendidos`, se delegó la lógica de análisis a un método privado genérico `procesarItem()`, evitando duplicar la iteración en los métodos `visitarProducto` y `visitarPaquete`.
5. **Estética del Modelo UML:** Se adoptó una convención visual estricta para garantizar la legibilidad del diagrama, utilizando colores específicos para clases de dominio (verde), abstractas (amarillo) y concretas de patrones (celeste), omitiendo métodos *helpers* internos para centrarse en contratos públicos de alto nivel.

6. Nota sobre la implementación de Pruebas Unitarias

Se deja constancia de que **no se han desarrollado tests unitarios sobre las interfaces del sistema** (como `MedioDePago`, `EstrategiaDeEnvio`, `Criterio`, etc.). Al tratarse puramente de contratos abstractos que no contienen lógica de dominio ejecutable ni estado interno, carecen de comportamiento susceptible de ser testeado mediante herramientas como JUnit. La cobertura del 95% exigida recae íntegramente sobre las clases concretas que implementan dichas interfaces.